

Generative AI Hackathon

3rd Edition 2026 – 27

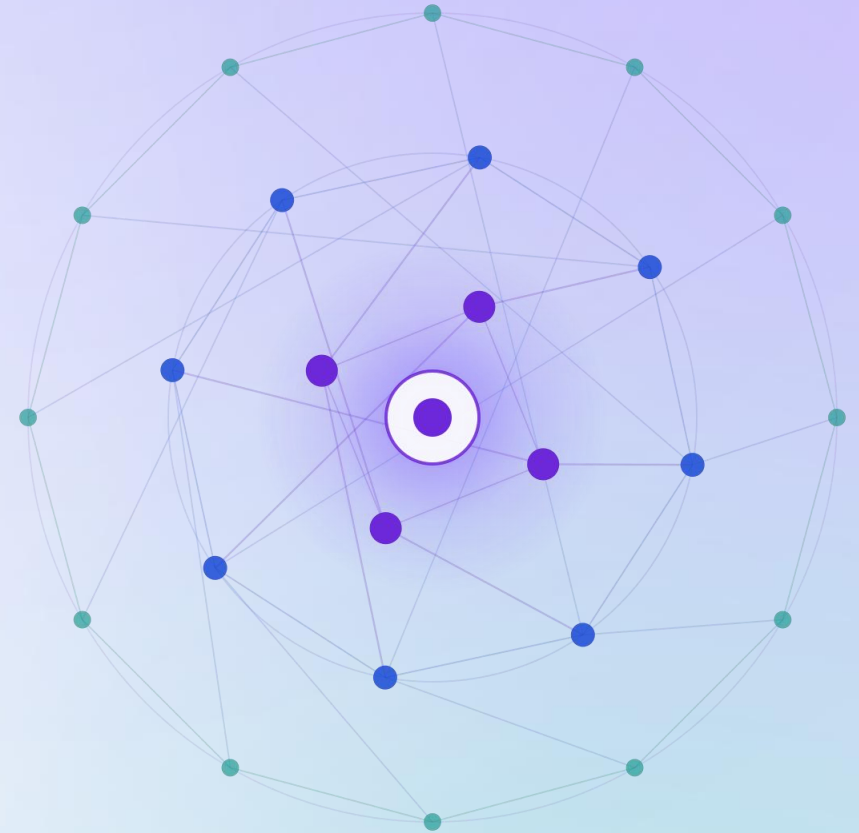
Five industry-defined themes. **One round** — ship a working prototype of multimodal, agentic AI that solves a real problem.

LAUNCH 11 SEP

SUBMIT 25 SEP

RESULTS 24 OCT

2 editions 600+ students 11 winning teams 5 IEEE publications



TRACK RECORD

Two editions in, and the numbers keep climbing

Edition 2 ran across 5 campuses in 4 cities and drew 848 idea submissions.

1ST EDITION · 2024 – 25

150+

Students

50+

Teams

6

Winning teams

5 IEEE publications · 4 Worklets · 2 summer internships, both converted to PPOs · VIT Vellore and VIT Chennai

2ND EDITION · 2025 – 26

848

Ideas submitted

120

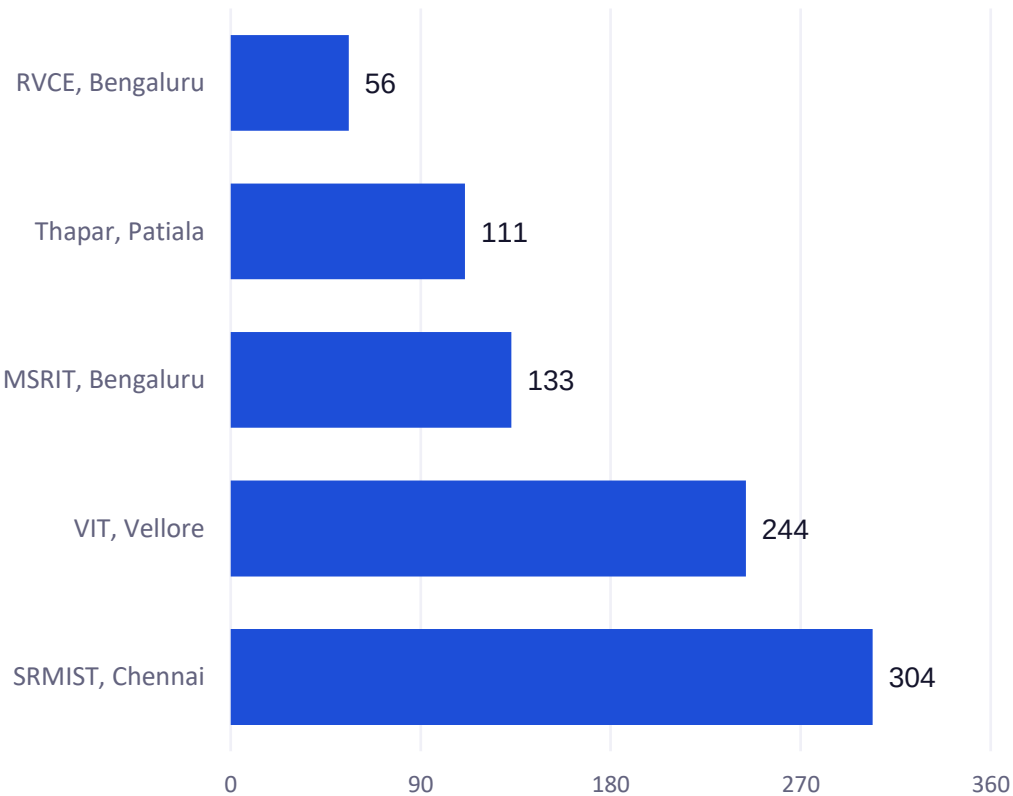
Teams shortlisted

5

Winning teams

466 students reached Round 2 · 14% shortlist rate · 5 campuses across 4 cities · 10 Worklets

Edition 2 — Round 1 ideas by campus



Five themes. Five real problems to solve.

Pick one theme, build one prototype. Four themes are confirmed below; the rest follow before launch.

01

THEME 01

Agentic Code Intelligence

Find the right code from a natural-language query.

02

THEME 02

Guided Troubleshooting

Turn a vague device complaint into one-click fix steps.

03

THEME 03

Teachable Voice Automation

Teach the assistant a flow once; it replays it on command.

04

THEME 04

Streaming Live RAG

Retrieve the right context mid-conversation, from one natural command.

05

THEME 05

Interruptible Real time Agents

Full duplex conversation agents with deliberative reasoning skills

Agentic Code Intelligence



Problem statement

Voice-assistant codebases are huge: dozens of agents and tools spread across thousands of files. A new developer cannot hold it all in their head, and neither can an LLM, since the whole repo will never fit in a context window. Finding where something happens, and then fixing it, is the slow part.

What to build

- Take a plain-English question and return the matching code snippets with file and line locations
- Answer structural queries: “which files call tool XYZ before tool ABC?”
- Answer usage queries: “where is the Bluetooth-settings deeplink used?”
- Work agentially: plan, search, read and refine over a codebase far larger than the context window
- Bonus: suggest optimisations for the code paths you surface

Scope & constraints

- Sample open-source codebase provided with the problem statement
- Single language: JavaScript
- Output is snippets and locations; optimisation suggestions are a bonus, full code generation is not required
- Must run on CPU; minimal GPU use allowed

Tech focus

- Code-aware embeddings and vector search, plus AST or call-graph indexing for structural queries
- Agentic retrieval loop over the index; report precision@k, recall, latency and indexing cost



Smart Guided Troubleshooting Engine



Problem statement

Customers describe device problems in vague language — "screen flickers and the battery dies fast". An agent then has to translate that into troubleshooting steps by hand, roughly 15 minutes per scenario across millions of calls, and the user still has to hunt through Settings to act on the advice.

What to build

- Query enrichment: normalise a raw complaint into a technical query
- A two-stage LLM engine that returns clean, ordered troubleshooting steps as JSON
- Map each step to the exact in-app Settings deeplink so the fix is one tap away
- A fast-path cache that serves pre-validated answers in under 300 ms

Scope & constraints

- Deliverable is a REST API returning structured JSON
- Mapping must stay reusable — the production target is 10k+ scenarios
- Deeplinks must logically resolve to the action steps.
- The output response json should include all actions , steps and associated deeplinks.

Tech focus

- LLM-based normalisation and step structuring; model choice is open
- Deeplink resolution and caching; report step accuracy, latency and cost per query



Teachable Voice Automation



Problem statement

Voice assistants cannot act inside third-party apps like Amazon or Zomato, so anything past first-party features falls back to a web result. Users already know which taps get the job done — they just have no way to hand that knowledge to the assistant.

What to build

- Let a user speak a command, then record the on-screen steps as they do it once
- Generalise that recording into a reusable flow, not a brittle tap replay
- Match later utterances, including paraphrases and changed values, to the learnt flow
- Replay reliably when screens change, and ask the user only when genuinely stuck

Scope & constraints

- Android; UI automation or accessibility services, not app-specific APIs
- Two or three third-party apps is enough to prove the idea
- At least one parameterised slot — item, quantity or address
- No credential capture; pause for payment and authentication

Tech focus

- Speech-to-intent, UI-tree understanding, flow synthesis from one demonstration
- Report learn success, replay success across sessions and paraphrase match accuracy



Streaming Live RAG



Problem statement

In a full-duplex conversation the user speaks one natural request, not a tidy query. A single utterance can hide several distinct questions, and the user has no idea how retrieval works, so they never phrase it for the retriever. Add a detail mid-flow and the answer has to get better, not start over.

What to build

- Intent understanding that decides whether an utterance needs retrieval at all
- Decompose one natural command into the several queries it actually implies
- Fuse and rerank results across sub-queries into a single grounded answer
- Use supplementary detail to sharpen the previous answer, not restart the search

Scope & constraints

- Full-duplex: begin retrieving before the utterance ends
- Corpus provided; voice input may be simulated from transcripts
- Session-scoped memory only — no cross-session user profile
- Simple and cheap beats a heavy agent orchestration stack

Tech focus

- Query decomposition, multi-query retrieval, rank fusion, reranking
- Report retrieval recall, answer groundedness, time-to-first-token and cost per turn



Interruptible Real-Time Agents



Problem statement

AI assistants today take turns: listen, think, then speak. People interrupt, change their mind mid-sentence and switch goals while a task is still running. Most systems either go silent while they reason, or throw everything away and start over. An assistant that stays responsive while it thinks, and recovers cleanly when the plan changes is an open problem.

What to build

- Keep user entertained and the conversation alive, without losing on logical depth and core functionality
- Handle and recover from real time interruptions, while driving the conversation towards end goal
- Cater to different changes in goals without losing the relevant session context.
- Make a solid harness around real-time agent to produce safe and reliable agentic structure.

Scope & constraints

- Full-duplex: begin retrieving before the utterance ends
- Corpus provided; voice input may be simulated from transcripts
- Session-scoped memory only — no cross-session user profile
- Out of scope: speech synthesis quality, wake-word detection, UI polish

Tech focus

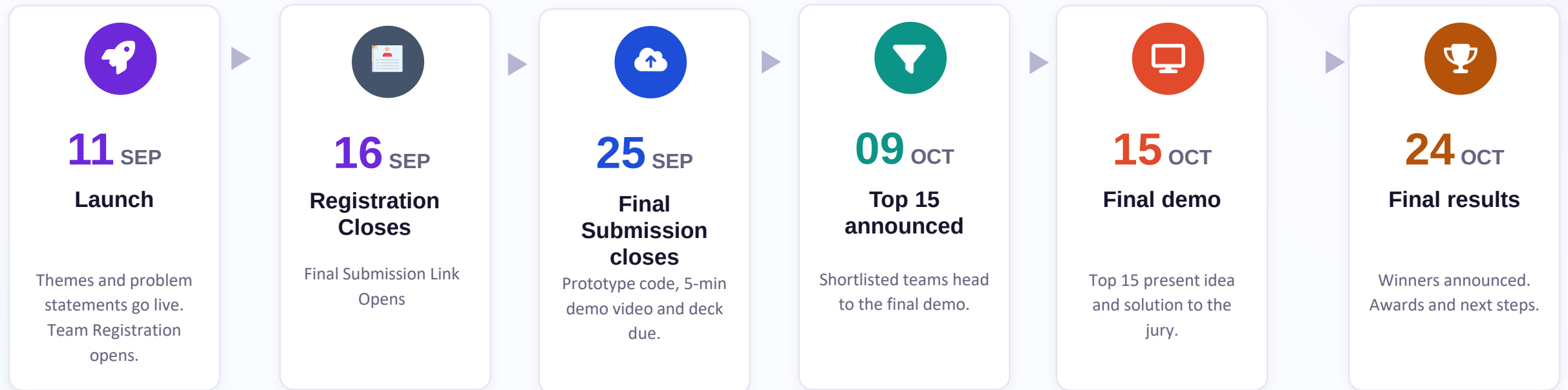
- Streaming and async agent designs
- Optimizing for latency, interruptions without compromising the reasoning depth and functions
- Multi-modal inputs and multimodal outputs.



TIMELINE

Six weeks, five milestones

11 September to 24 October 2026. Dates are tentative and may shift slightly.



Build & submit 11 – 25 Sep, 2 weeks

Evaluation 25 Sep – 9 Oct

Final demo round 15 Oct, top 15 teams

**** Final Submission Link will be ONLY shared with Teams who have completed the **Team registration process****

What's in it for you



Certificates

Every *shortlisted team receives a participation certificate; winners get a merit certificate.



Prizes for the top teams worth ₹1.5 L

Awards for the winning teams at the final results announcement on 24 October.



PRISM worklet

**Winning teams get an opportunity to work in PRISM worklet with a Samsung mentor.



Mentorship that counts

Kick-off session and ongoing guidance from Samsung Language AI and Tech Strategy engineers.



Publish your work

Edition 1 winners produced 5 IEEE conference and journal publications from their hackathon projects.



Internship at Samsung R&D

Two-month summer internship for ***selected students. In Edition 1, both interns converted to pre-placement offers.

** Shortlisted – Team which will appear for final demo*

*** Based on the availability of the mentors, students & problem statement*

****Selected students – may have to appear for MAGPIE Coding test*

FORMAT

One build round. One final demo.

No ideation-only stage this year — Round 1 asks for a working prototype from day one.



SUBMISSION

Build & submit

11 – 25 Sep · open to all teams

What you submit

- Working prototype code — GitHub repo with README
- Demo video, max 5 minutes (YouTube or Drive link)
- Presentation file (PPT or PDF)
- Readme, docker & other requirement
- Submitted through the Google Form

Evaluation

Working prototype & functionality	30%
Technical depth & feasibility	25%
Innovation & originality	20%
Relevance to theme	15%
Presentation & documentation	10%

TOP 15 TEAMS SHORTLISTED · 9 OCT



FINAL DEMO ROUND

Present to the jury

15 Oct · top 15 teams only

What happens

- Present your idea and solution live
- Walk the jury through the working prototype
- Q&A on design decisions and trade-offs
- Results announced on 24 October

What the jury looks for

Does the prototype actually work?
Is the technical approach sound?
Would a real user want this?
Can it be taken further as a worklet?

WINNERS & AWARDS · 24 OCT

Guideline for Registration (Last date – 16th Sep 2026, 11:59 PM)

Google Form



Team rules

Up to max 4 members

per team, single college

Team name

CollegeName_TeamName

Per member

name, email, phone, year & branch

One theme

one submission per team

Guideline for FINAL Submission (Last date – 25th Sep 2026, 11:59 PM)

Everything below needs to be submitted by 25 September. There is no separate ideation submission.

Google Form



What the submission covers

- Working prototype code - public or shared GitHub repo
- Demo video, max 5 minutes (YouTube or Drive link)
- Presentation file (PPT or PDF)- **CollegeName_TeamName** Follow the nomenclature
- One submission per team via the Google Form

**** Teams NOT following the submission guideline would lead to **direct disqualification****

GitHub Submission (Last date – 25th Sep 2026, 11:59 PM)

Google Form

☐ The GitHub Submission

- Working prototype code : public or shared.
- README with reproducible setup instructions, Docker files, and other requirements
- For your final submission: Create a release tag named **PRISM_GENAI_HACKATHON_Y2026** on your final commit. The tagged commit is what gets judged
- Make sure everything referenced in your submission—PPT, demo video, documentation, etc.—is present in the tagged commit

Google Form

☐ The PPT/Demo Video

- Refer to the **CollegeName_TeamName_Submission_ppt**
- Theme ID, project title and team details
- Problem statement in your own words
- Solution and architecture diagram
- Tools and tech stack used
- Innovation highlights, results and limitations

**** Teams NOT following the submission guideline would lead to **direct disqualification****

Ready to build?

Registrations open 11 September 2026.

Pick a theme, form a team of up to four, and submit a working prototype with a 5-minute demo video by 25 September. The top 15 teams present live to a Samsung R&D jury.



Registration link

<https://forms.gle/NxN6TWXLpcXmTnv66>



Queries

prism@samsung.com

Organised by the Language AI Team and the PRISM Team, Samsung R&D Institute India

KEY DATES

- **11 Sep** Launch
- **16 Sep** Registration closes
- **25 Sep** Prototype submission due
- **9 Oct** Top 15 announced
- **15 Oct** Final demo round
- **24 Oct** Final results

Tentative — subject to minor changes.

THANKYOU